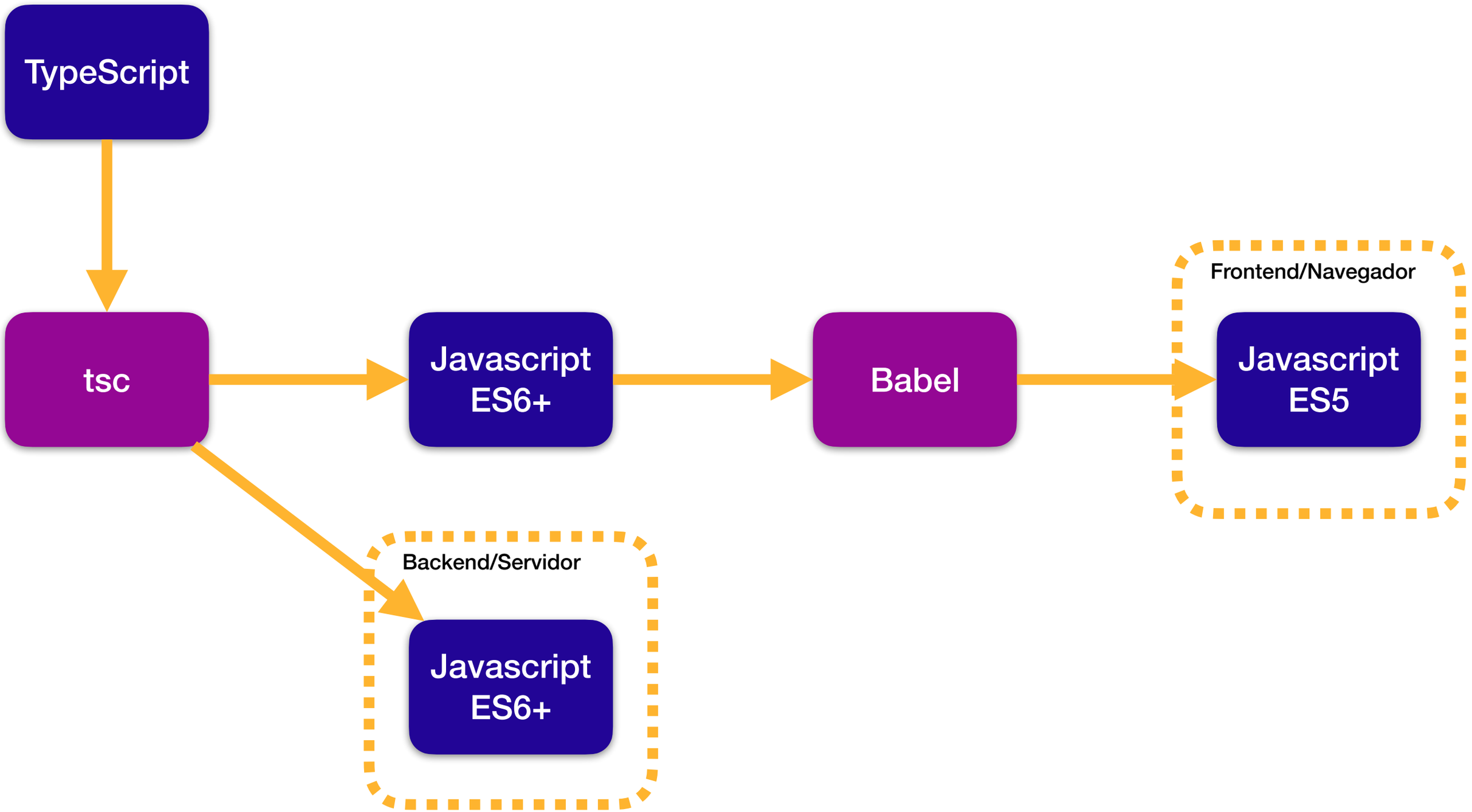
PROGRAMANDO UN SERVIDOR

Matias Toro

Slides inspiradas de https://fullstackopen.com/en/part3

Node.js y Express

Versiones de Javascript



Creando un servidor desde cero

Desde una carpeta nueva

```
npm init
npm install --save-dev typescript @types/node ts-node-dev
npx tsc --init
```

package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "main": "index.ts",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": ""
}
```

El script inicial es
index.ts

Corriendo archivos TypeScript

tsconfig.json

```
  "compilerOptions": {
    …
    "target": "es2016",
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./out",
    "strict": true,
    …
}
```

package.json

```
  "scripts": {
    "build": "tsc",
    "start": "node out/index.js",
    "dev": "ts-node-dev src/index.ts",
    "test": "echo \"Error: …"
  }
```

src/index.ts

```
console.log("Hola Mundo");
```

Correr directamente

```
npx ts-node src/index.ts
```

Compilar y correr

```
npm run build
npm run start
```

Corriendo archivos TypeScript 7+

tsconfig.json

```
"compilerOptions": {
  …
  "target": "nodenext",
  "module": "esnext",
  "rootDir": "./src",
  "outDir": "./out",
  "strict": true,
  …
  "allowImportingTsExtensions": true,
  "rewriteRelativeImportExtensions": true,
}
```

package.json

```
  "scripts": {
    "build": "tsc",
    "start": "node out/index.js",
    "dev": "node --watch src/index.ts",
    "test": "echo \"Error:.."
  },
  "type": "module"
```

src/index.ts

```
console.log("Hola Mundo");
```

Correr directamente

```
node src/index.ts
```

Utilizando Express

```
npm install express
npm install --save-dev @types/express
```

```
"dependencies": {
  "express": "^5.1.0"
}
```

“Al menos” 5.1.0
$(\geq 5.1.0, \ < 6.0.0)$

Actualiza las versiones de las dependencias (mismas restricciones)

```
npm update
```

Instala todo lo que esta declarado en `package.json`

```
npm install
```

Servidor web usando express

Las direcciones serán relativas a `src` cuando es claro del contexto

index.ts

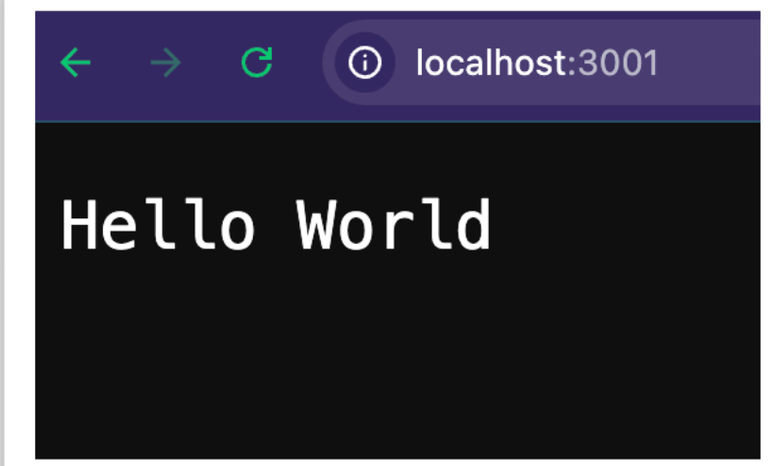
```ts
import express from "express";
const app = express();

app.get("/", (request, response) => {
    response.send("<h1>Hello World!</h1>");
});



const PORT = 3001;
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

```
Server running on port 3001
```



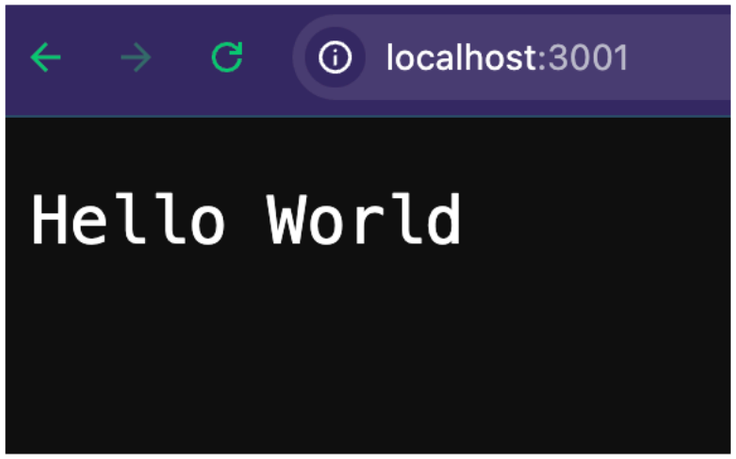
Retornando un JSON

```
const notes = [
    {
        id: "1",
        content: "HTML is easy",
        important: true,
    },
    {
        id: "2",
        content: "Browser can execute only JavaScript",
        important: false
    },
    {
        id: "3",
        content: "GET and POST are the most important methods of HTTP protocol",
        important: true
    }
];


app.get("/api/notes", (req, res) => {
  req.json(notes);
});
```

No necesitamos llamar a JSON.stringify, Express transforma los datos automáticamente

El *content-type* es ajustado automáticamente



reiniciar



Observando cambios automáticamente

```
npm run dev
```

Correrá el programa en modo “recarga automáticamente si detectas cambios”

```
npx ts-node-dev src/index.ts
```

O directamente…

```
node --watch src/index.ts
```

En ts7

RESTful HTTP API

- Asumamos que la url de nuestro servicio es http://www.example.com/api

- Si nuestro recurso son notas, entonces la dirección de una nota con identificador 10 es: http://www.example.com/api/notes/10

- La url de la colección de todos los recursos es: http://www.example.com/api/notes

RESTful HTTP API

Operaciones en recursos

URL	Método	Funcionalidad
notes/10	GET	obtiene un único recurso
notes	GET	obtiene todos los recursos de la colección
notes	POST	crea un nuevo recurso a partir de los datos de la solicitud
notes/10	DELETE	elimina el recurso identificado
notes/10	PUT	reemplaza completamente el recurso identificado con los datos de la solicitud
notes/10	PATCH	reemplaza parcialmente el recurso identificado con los datos de la solicitud

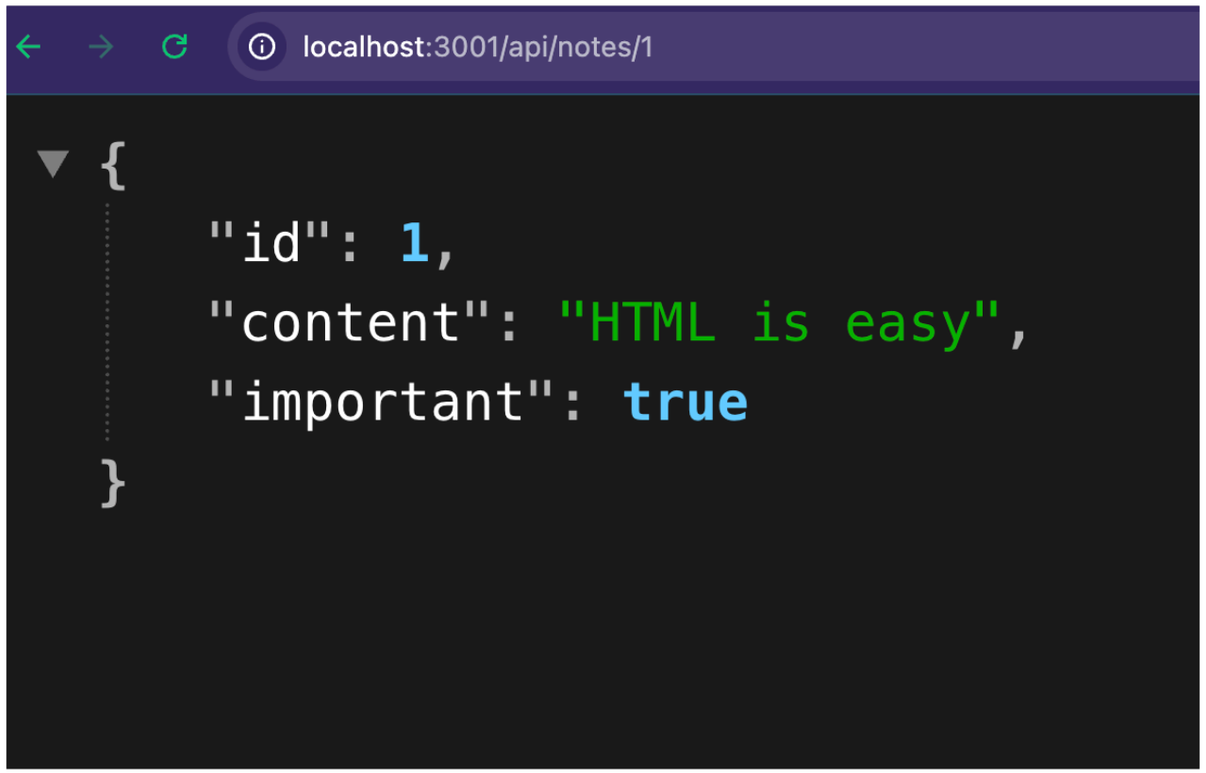
Obteniendo un único recurso

Definimos **parametros** en rutas de Express usando “:”

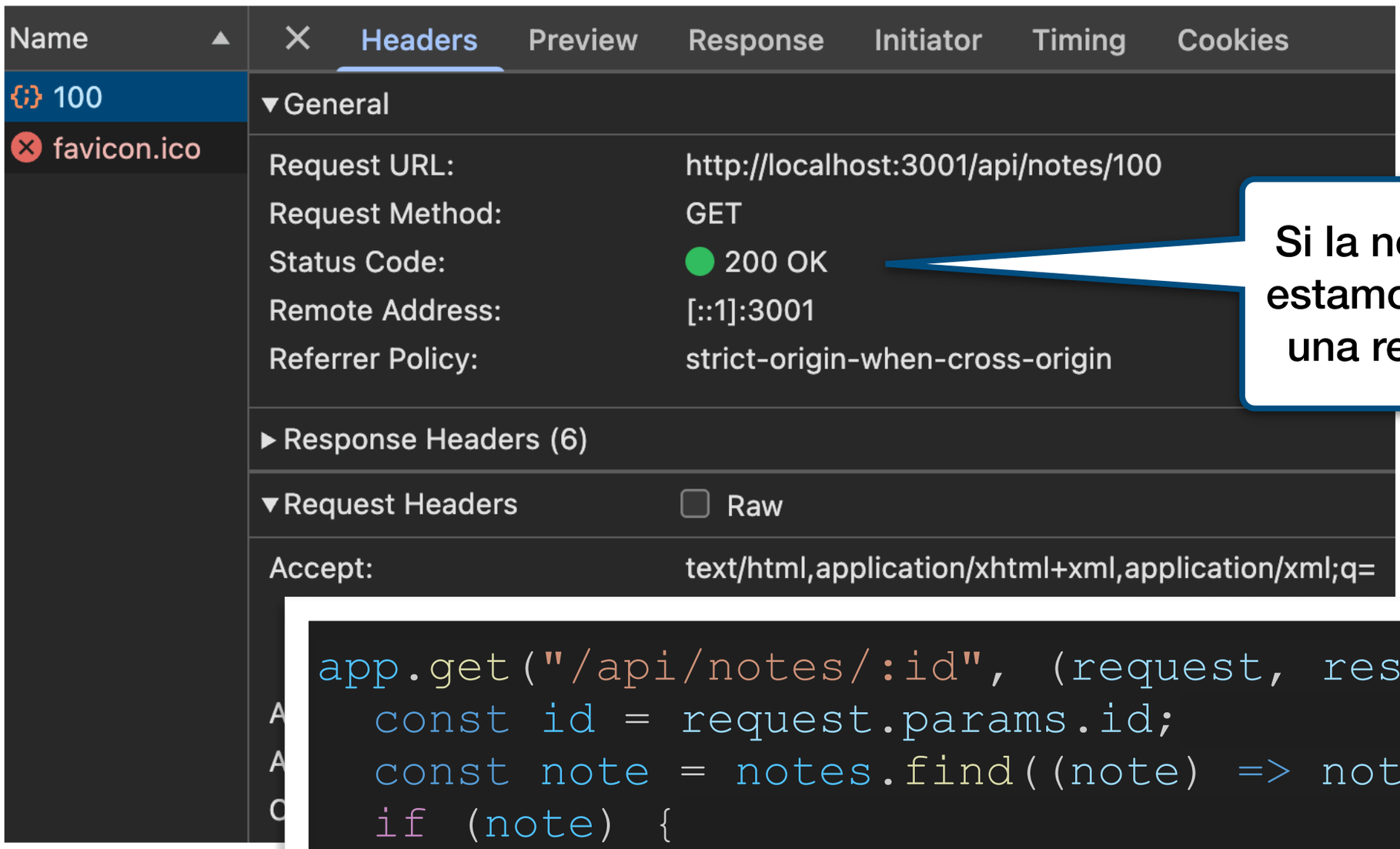
Hace match con rutas del estilo `/api/notes/foobar`

```
app.get("/api/notes/:id", (request, response) => {
  const id = request.params.id;
  const note = notes.find((note) => note.id === id);
  response.json(note);
});
```

El parámetro id lo podemos obtener a través del objeto request



Obteniendo un único recurso



Si la nota no existe, estamos obteniendo una respuesta 200

```
app.get("/api/notes/:id", (request, response) => {
  const id = request.params.id;
  const note = notes.find((note) => note.id === id);
  if (note) {
    response.json(note);
  } else {
    response.status(404).end();
  }
});
```

Agregamos un caso para cuando no existe la nota

Eliminando recursos

Cambiamos `const` por `let` para poder modificar `notes`

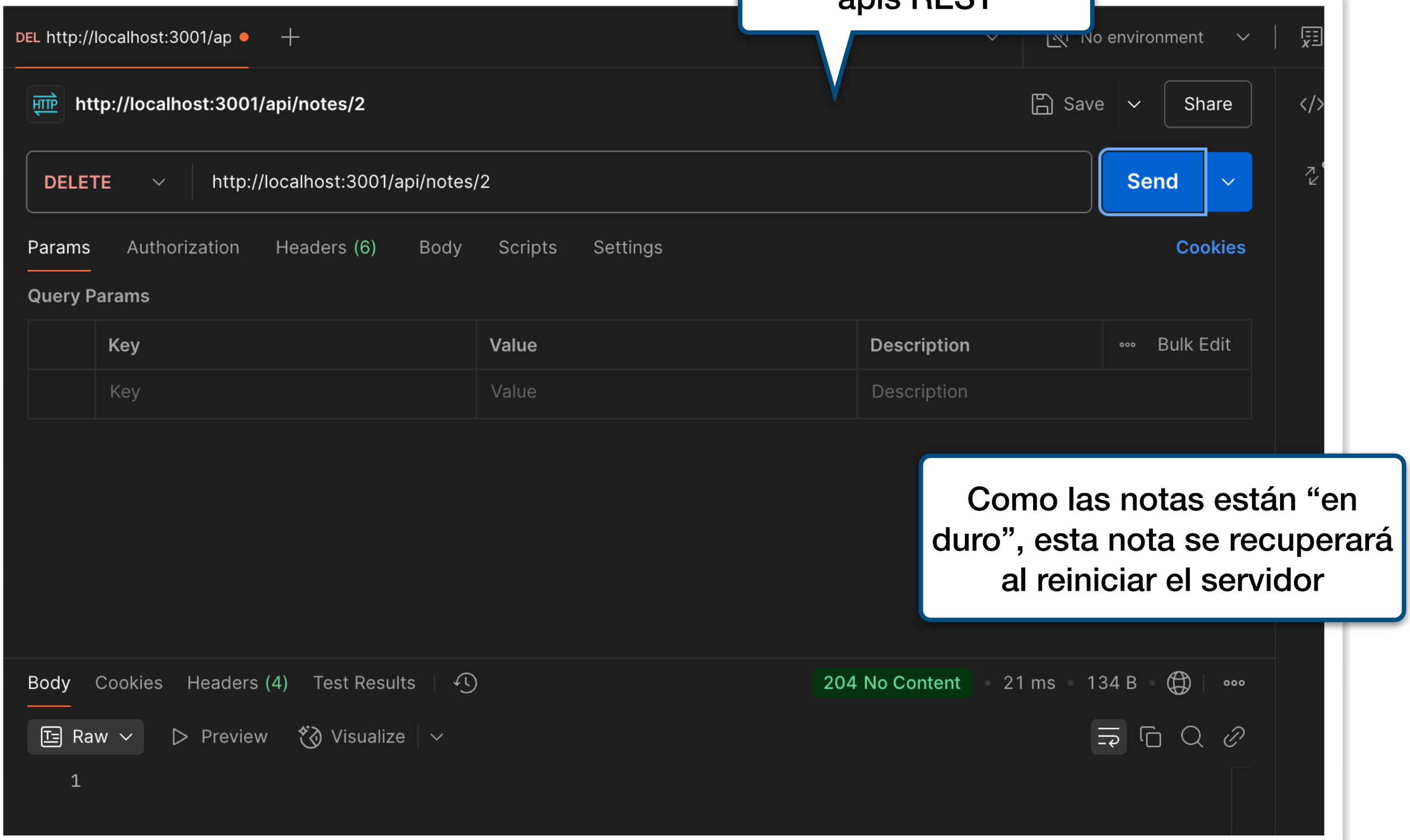
```
let notes = [
…
app.delete("/api/notes/:id", (request, response) => {
  const id = request.params.id;
  notes = notes.filter((note) => note.id !== id);

  response.status(204).end();
});
```

Respondemos con 204 para decir “éxito” pero no hay contenido que mostrar

Como probamos este nuevo endpoint?

Usando Postman



Usando VSCode Rest Client

Creamos un archivo `requests.rest` en la raíz de nuestro directorio

Hacemos click en Send Request para ejecutar la llamada

Podemos hacer prepend del tipo de request si queremos: DELETE http://localhost…

La respuesta del servidor



Recibiendo datos

```
…
const app = express();

app.use(express.json());

let notes = [
…
app.post("/api/notes", (request, response) => {
  const note = request.body;
  console.log(note);
  response.json(note);
});
```

Le decimos a Express que parsee JSONs

Entonces, el objeto JSON será parseado y dispuesto en la propiedad `body`

Probando en Postman

POST http://localhost:3001/a

No environment

http://localhost:3001/api/notes

Save | Share

POST | http://localhost:3001/api/notes | Send

Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Beautify

```
1  {
2      "content": "Estamos testeando un POST",
3      "important": true
4  }
```

```
Server running on port 3001
{ content: 'Estamos testeando un POST', important: true }
```

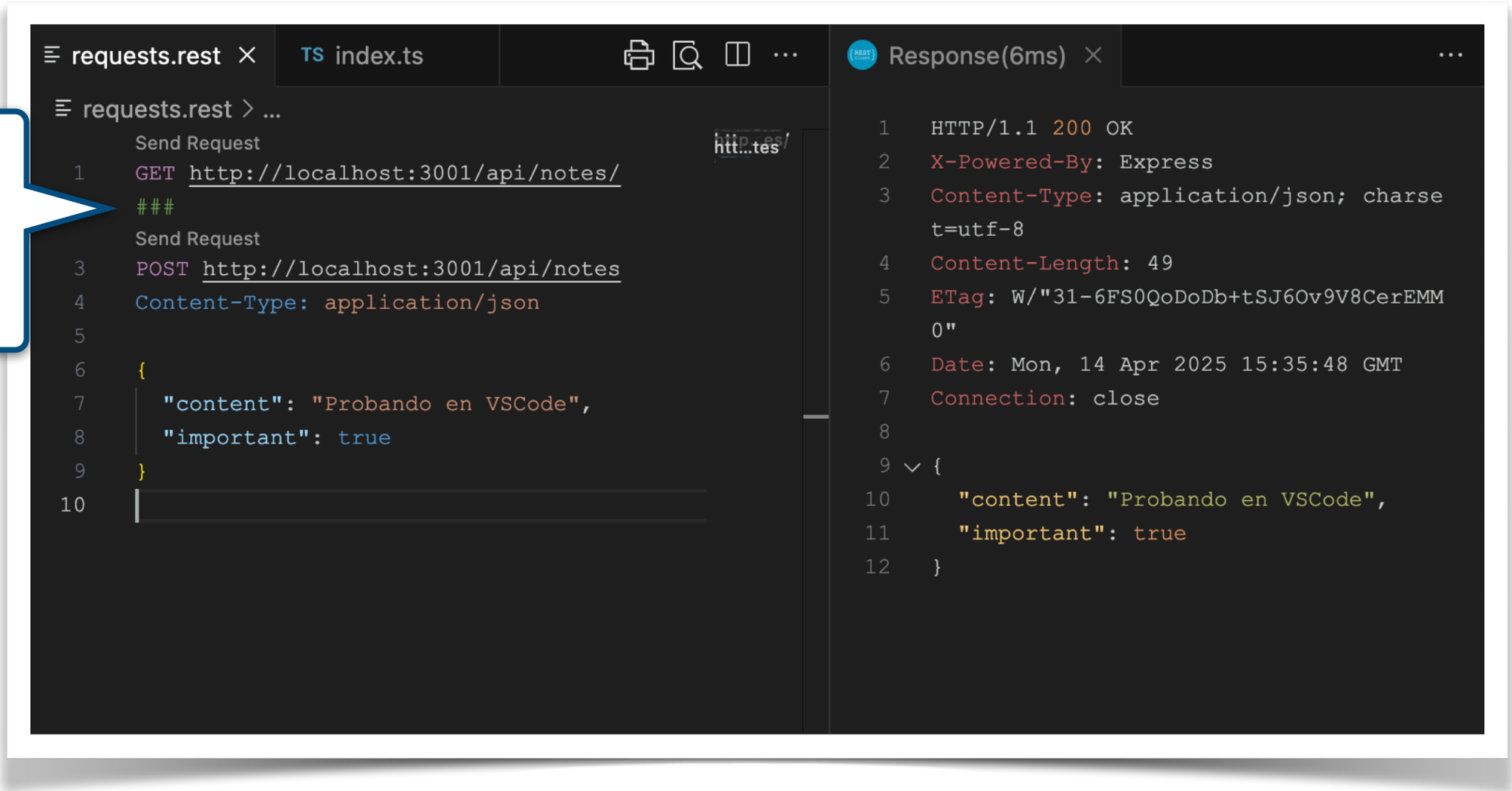
Body | Cookies | Headers (7) | Test Results | 200 OK | 22 ms | 291 B

{} JSON | Preview | Visualize

```
1  {
2      "content": "Estamos testeando un POST",
3      "important": true
4  }
```

Probando en VSCode

Usamos ### para separar requests



Finalizando el método POST

```
app.post("/api/notes", (request, response) => {
  const maxId =
    notes.length > 0 ?
      Math.max(...notes.map((n) => Number(n.id))) : 0;

  const note = request.body;
  note.id = maxId + 1;

  notes = notes.concat(note);

  response.json(note);
});
```

Pero acepta cualquier cosa!. Al menos queremos que tenga la propiedad `content`

Finalizando el método POST

```
const generateId = () => {
  const maxId = notes.length > 0 ?
                    Math.max(...notes.map((n) => Number(n.id))) : 0;
  return (maxId + 1).toString();
};

app.post("/api/notes", (request, response) => {
  const body = request.body;

  if (!body.content) {
    response.status(400).json({
      error: "content missing",
    });
  } else {
    const note = {
      content: body.content,
      important: body.important || false,
      id: generateId(),
    };

    notes = notes.concat(note);

    response.json(note);
  }
});
```

Hemos extraído la lógica que genera un nuevo id

Devolvemos un error si no viene la propiedad content

Si la propiedad “important” no viene, entonces es `false` por defecto

Los Middleware

- El json-parser de Express es un *middleware*
- Los middleware son funciones que se pueden usar para manejar los objetos de solicitud (request) y respuesta (response).

```
import express, {
    type NextFunction,
    type Request,
    type Response,
} from "express";
const app = express();

app.use(express.json());

const requestLogger = (
  request: Request,
  response: Response,
  next: NextFunction
) => {
  console.log("Method:", request.method);
  console.log("Path:  ", request.path);
  console.log("Body:  ", request.body);
  console.log("---");
  next();
};
app.use(requestLogger);
…
```

Podemos crear nuestro propio middleware

Este se ejecutará después de json-parser

Los Middleware

Los middleware deben ser usados antes de la definición de las rutas que los usan

```
const unknownEndpoint = (request: Request, response: Response) => {
  response.status(404).send({ error: "unknown endpoint" });
};

app.use(unknownEndpoint);
```

Si lo hacemos después de definir las rutas, va a ser usado por cualquier otra ruta. Útil para personalizar las rutas indefinidas

Desplegando la aplicación

Conectando el frontend con el backend

Frontend: src/services/notes.tsx

```
import axios from "axios";
import { NoteData } from "../types/data";
const baseUrl = "http://localhost:3001/api/notes";

const getAll = () => {
...
```

```
render 0 notes                                          App.tsx:20
Access to XMLHttpRequest at '                           localhost/:1
http://localhost:3001/api/notes' from origin '
http://localhost:5173' has been blocked by CORS
policy: No 'Access-Control-Allow-Origin' header is
present on the requested resource.
▶ GET                                                   notes.tsx:6
http://localhost:3001/api/notes
net::ERR_FAILED 200 (OK)
▶ Uncaught (in promise)                    axios.js?v=9468f60c:1580
  AxiosError {message: 'Network Error', name: 'Axios
  Error', code: 'ERR_NETWORK', config: {…}, request:
  XMLHttpRequest, …}
```

Solicitud bloqueada por política de CORS

Política del mismo origen y CORS

- **Same-Origin Policy**: Los navegadores implementan esta política de seguridad para evitar vulnerabilidades como el secuestro de sesión; solo permiten que páginas web interactúen con recursos del mismo origen (mismo esquema, host y puerto).

- **CORS (Cross-Origin Resource Sharing)**: Es un mecanismo creado para permitir solicitudes legítimas entre diferentes orígenes; el navegador verifica la cabecera Access-Control-Allow-Origin para aceptar o rechazar respuestas.

- **Problema típico en desarrollo**: Cuando el frontend y el backend están en diferentes puertos (por ejemplo, 5173 y 3001 en localhost), no comparten el mismo origen, por lo que es necesario configurar **CORS** para permitir la comunicación.

Instalando CORS en el backend

```
npm install cors
npm install --save-dev @types/cors
```

```
import cors from "cors";

const app = express();

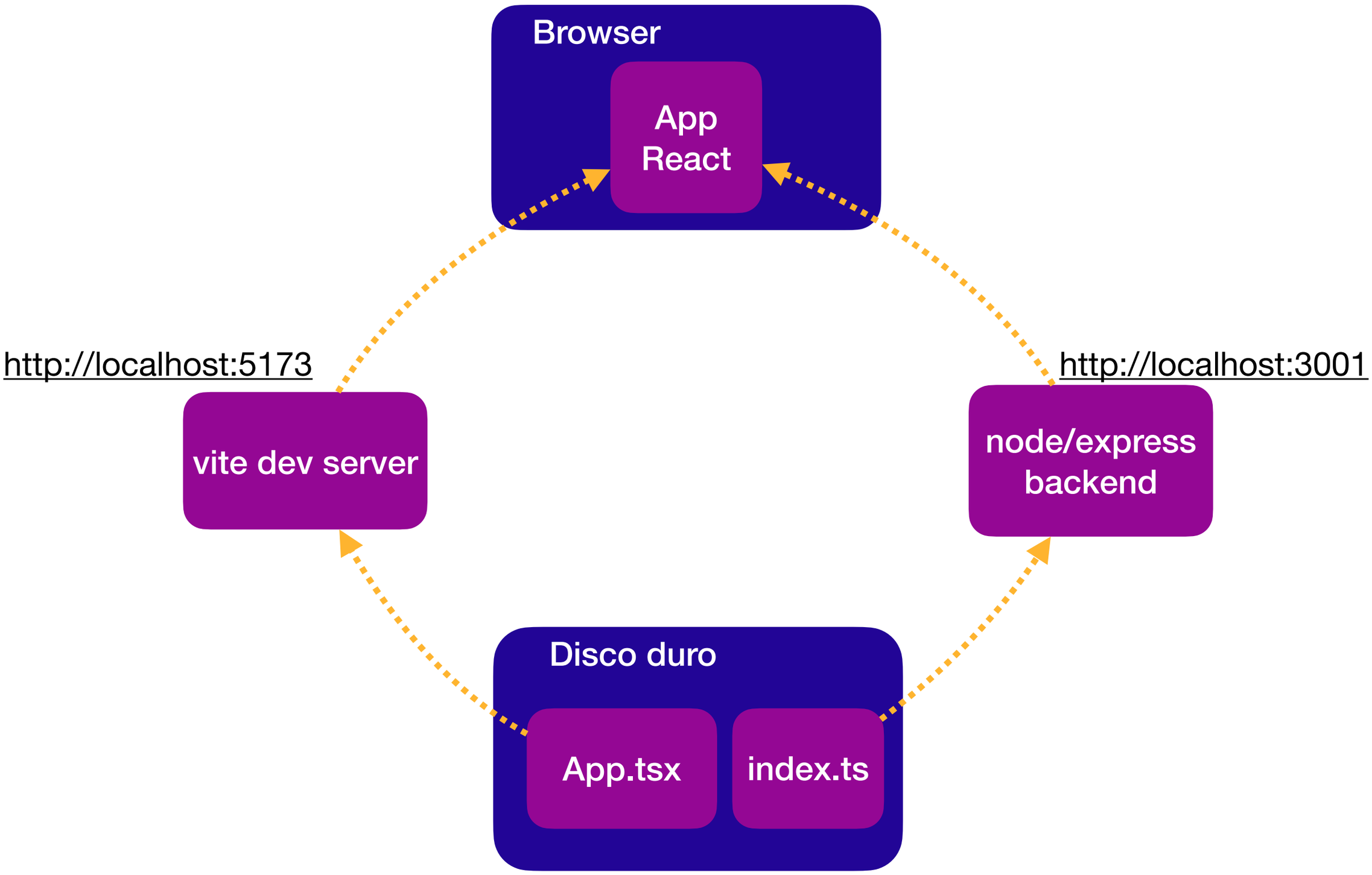
app.use(express.json());
app.use(cors());
…
```

Podemos usar algo como

```
app.use(cors({
  origin: 'http://localhost:5173',
}));
```

para solo permitir solicitudes de esa url

El ambiente de desarrollo



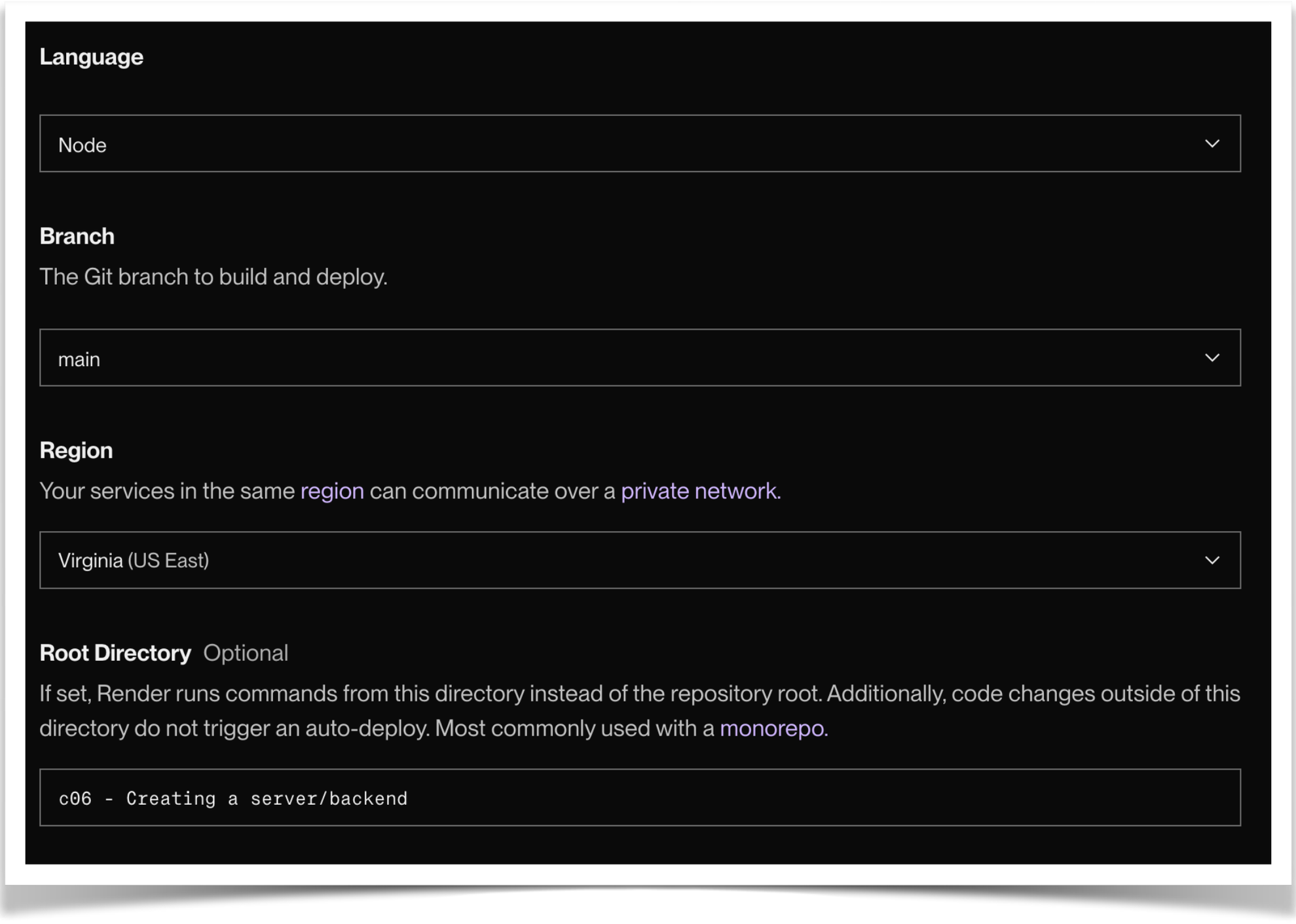
Poniendo en producción el backend

Backend: cambiamos `index.ts` para que el puerto se lea del ambiente

index.ts

```ts
const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Usando Render



Usando Render

Build Command

Render runs this command to build your app before each deploy.

```
c06 - Creating a server/backend/ $ npm install && npm run build
```

Start Command

Render runs this command to start your app with each deploy.

```
c06 - Creating a server/backend/ $ npm run start
```

Instance Type

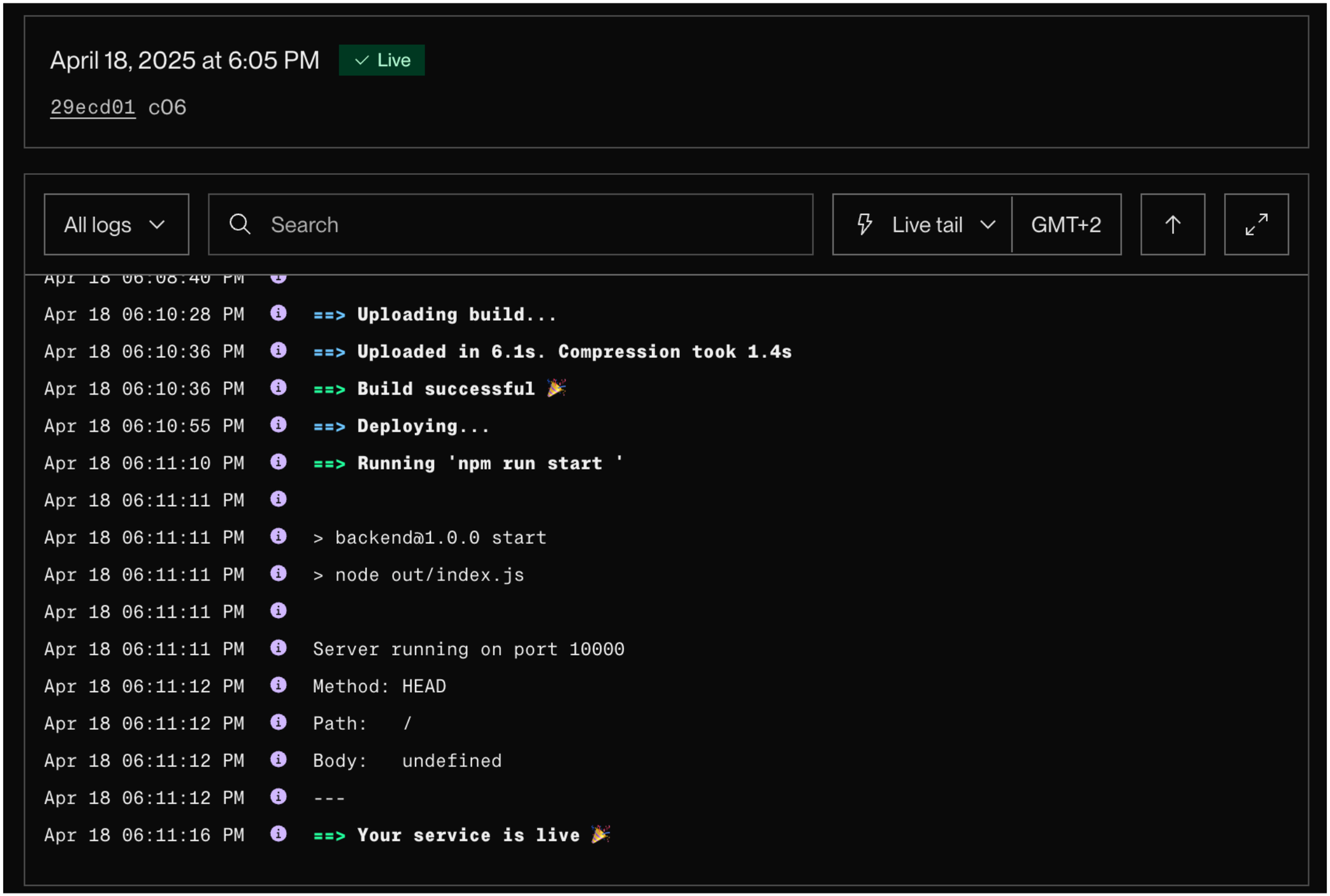
For hobby projects

Free	512 MB (RAM)
$0 / month	0.1 CPU

⚠ **Upgrade to enable more features**

Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

Usando Render



Build de producción del frontend

- Hasta ahora hemos corrido React en modo desarrollo
- Para obtener una versión de producción necesitamos correr el comando `npm run build`

```
npm run build

> communication@0.0.0 build
> tsc -b && vite build

vite v6.2.4 building for production...
✓ 83 modules transformed.
dist/index.html                   0.46 kB │ gzip:  0.30 kB
dist/assets/index-CHxXcJq5.css    0.20 kB │ gzip:  0.15 kB
dist/assets/index-C2gEM7KZ.js   224.62 kB │ gzip: 73.72 kB
✓ built in 430ms
```

Build de producción del frontend

```
npm run build

> communication@0.0.0 build
> tsc -b && vite build

vite v6.2.4 building for production...
✓ 83 modules transformed.
dist/index.html                   0.46 kB │ gzip:  0.30 kB
dist/assets/index-CHxXcJq5.css    0.20 kB │ gzip:  0.15 kB
dist/assets/index-C2gEM7KZ.js   224.62 kB │ gzip: 73.72 kB
✓ built in 430ms
```

Código compilado a ES6 y minificado

```
(function(){const
f=document.createElement("link").relList;if(f&&f.supports&&f.supports("modulepreload"))return
;for(const d of document.querySelectorAll('link[rel="modulepreload"]'))s(d);new
MutationObserver(d=>{for(const y of d)if(y.type==="childList")for(const S of
y.addedNodes)S.tagName==="LINK"&&S.rel==="modulepreload"&&s(S)}).observe(document,
{childList:!0,subtree:!0});function o(d){const y={};return
d.integrity&&(y.integrity=d.integrity),d.referrerPolicy&&(y.referrerPolicy=d.referrerPolicy),
d.crossOrigin==="use-credentials"?y.credentials="include":d.crossOrigin==="anonymous"?
y.credentials="omit":y.credentials="same-origin",y}function s(d){if(d.ep)return;d.ep=!0;const
y=o(d);fetch(d.href,y)}})();var Of={exports:{}},Du={};
…
```

Sirviendo archivos estáticos desde el backend

```
cp -r dist ../backend
```

Backend: Cada ves que llegue una solicitud HTTP GET, mirará el directorio `dist` si es que hay un archivo con ese nombre

```
const app = express();

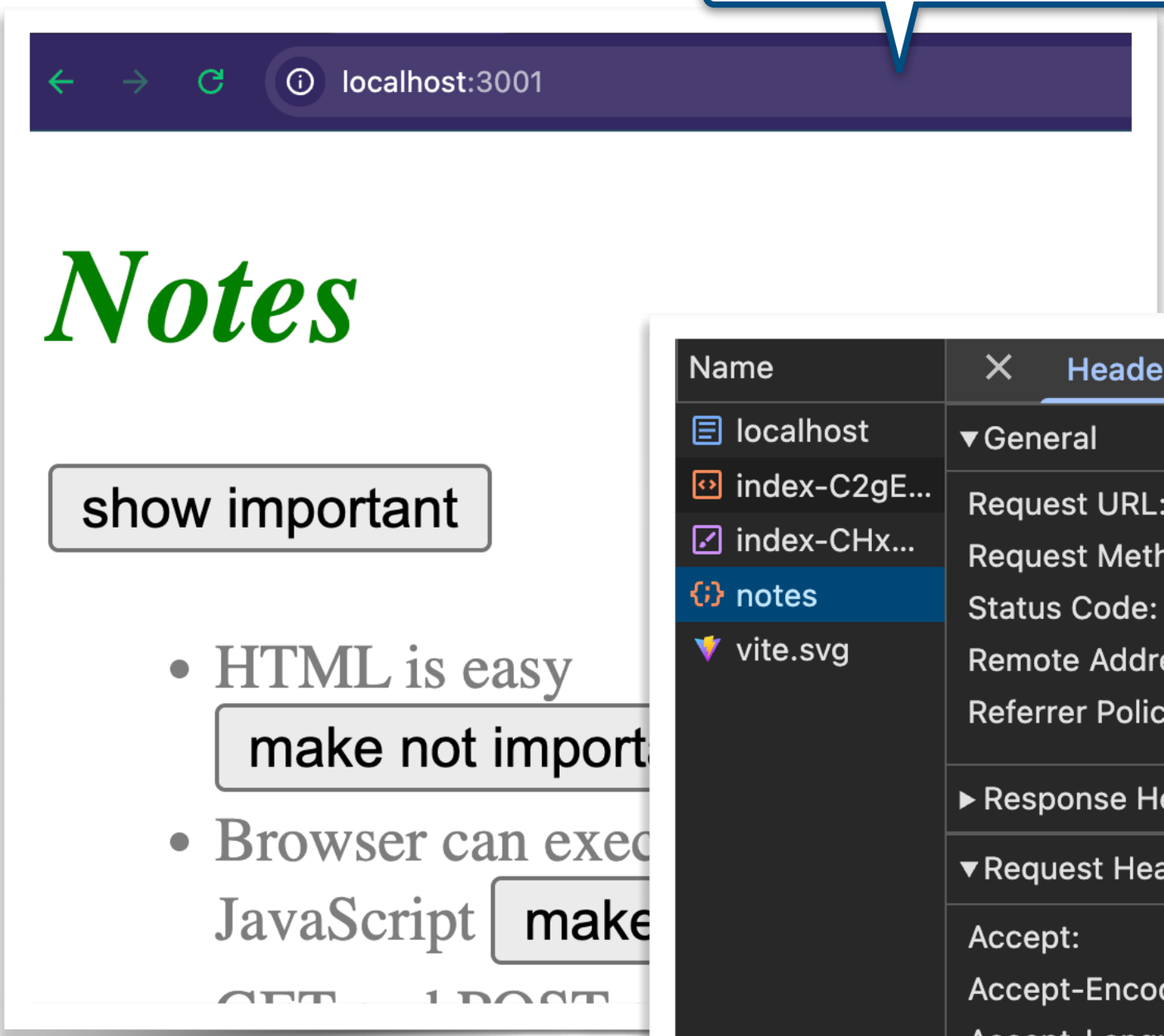
app.use(express.json());
app.use(cors());
app.use(express.static("dist"));
```

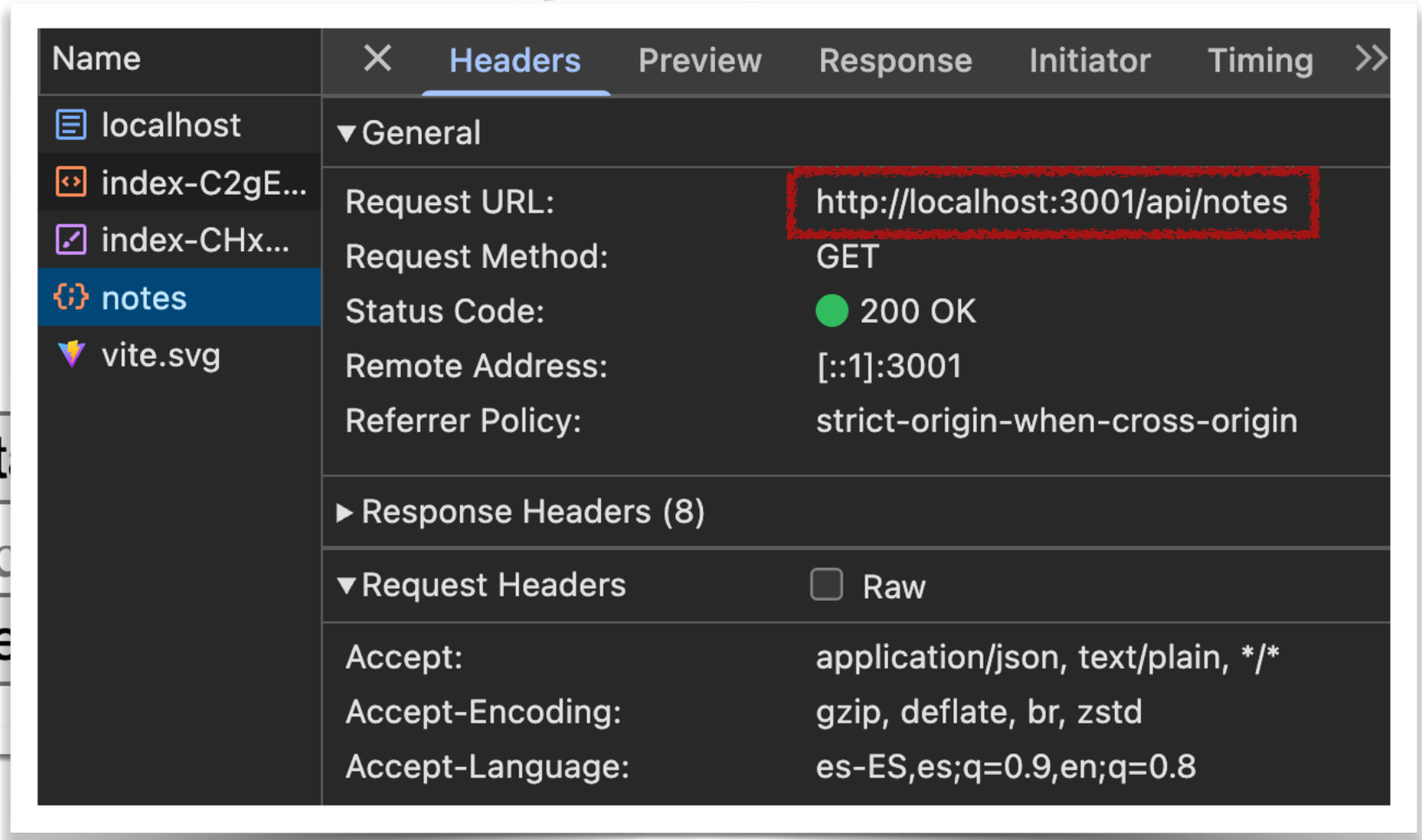
Frontend: Ahora el frontend y el backend tienen la misma url por lo que la url base la seteamos como una dirección relativa

```
import axios from "axios";
import { NoteData } from "../
types/data";
const baseUrl = "/api/notes";
```

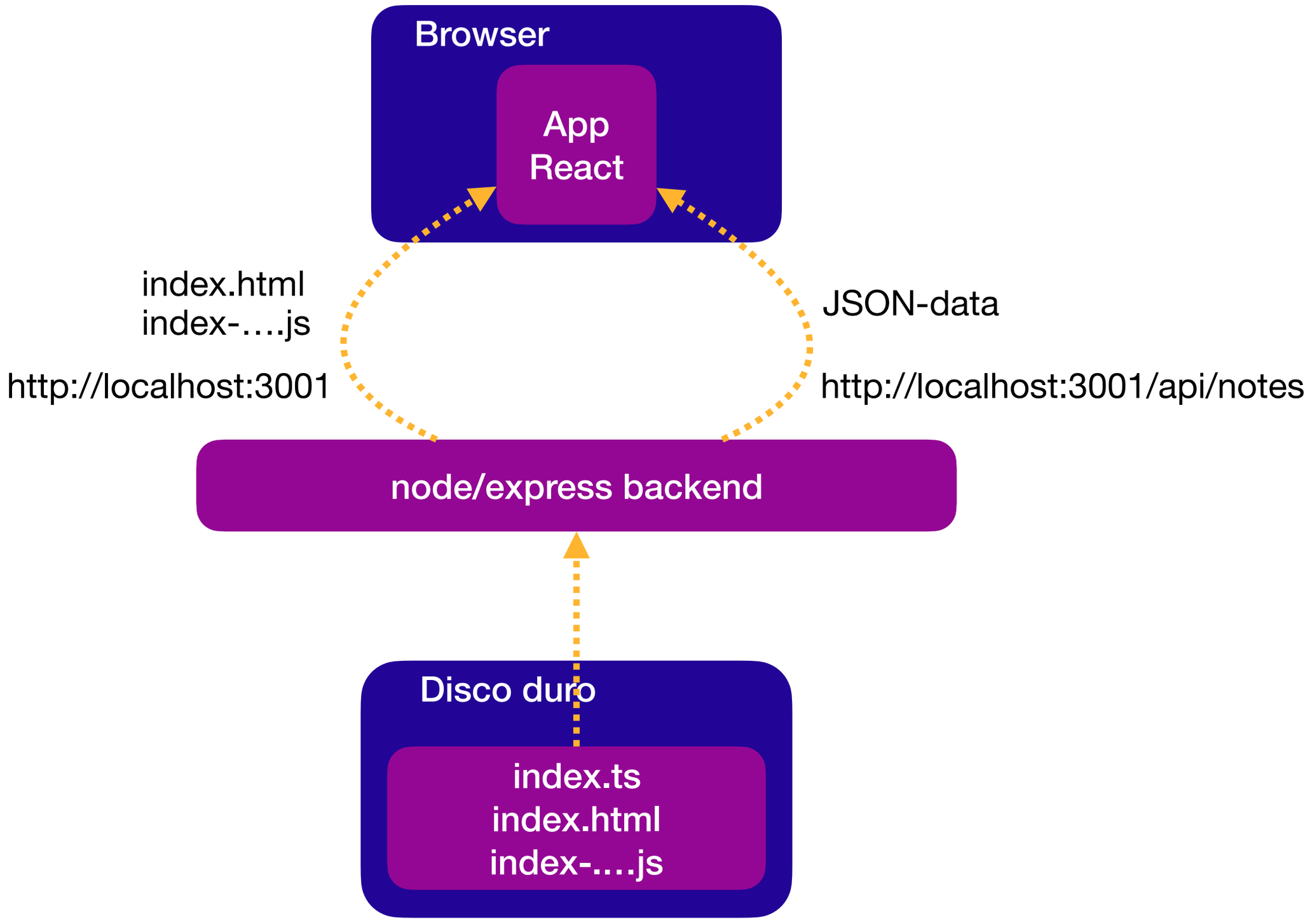
Sirviendo archivos estáticos desde el backend

http://localhost:3001/

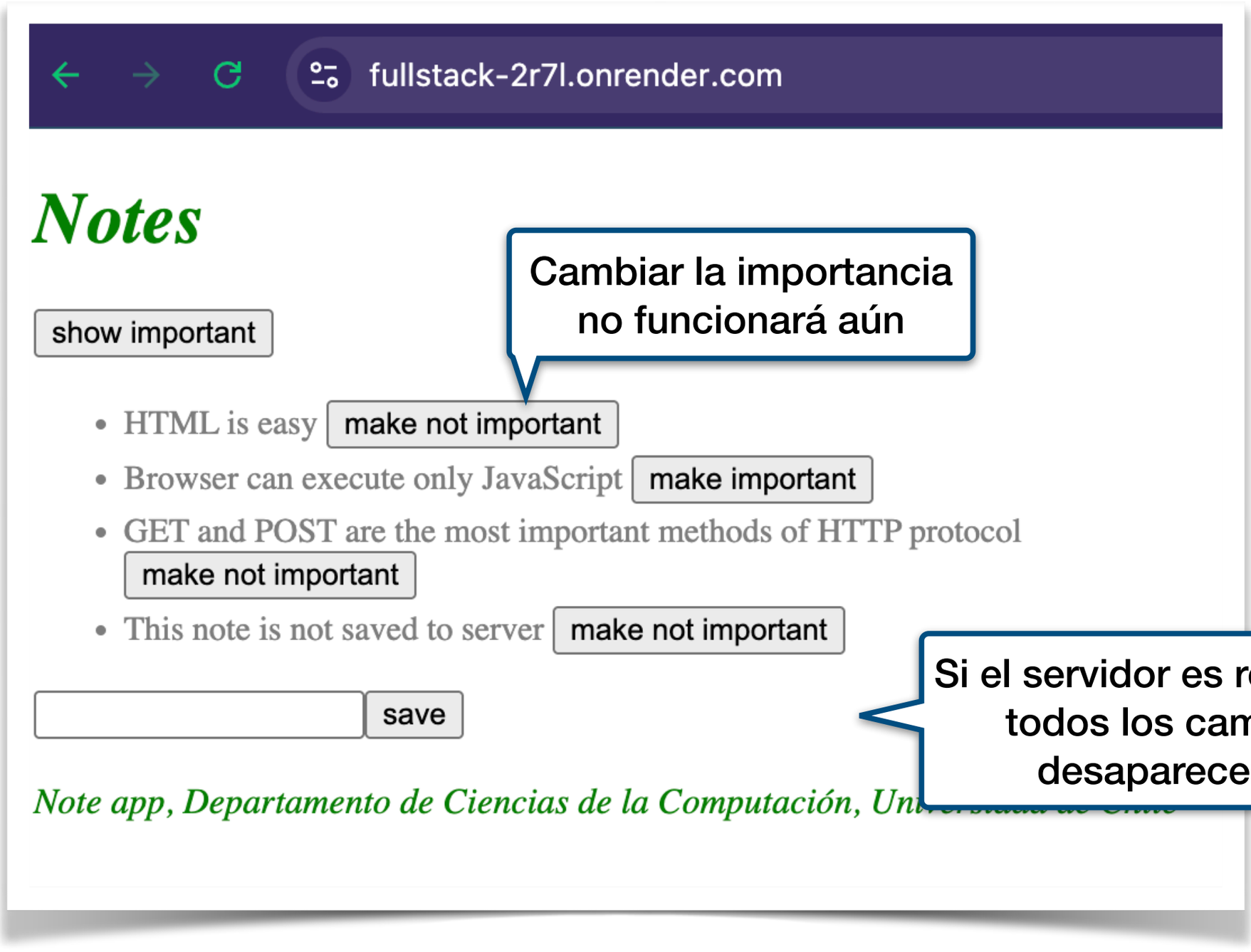




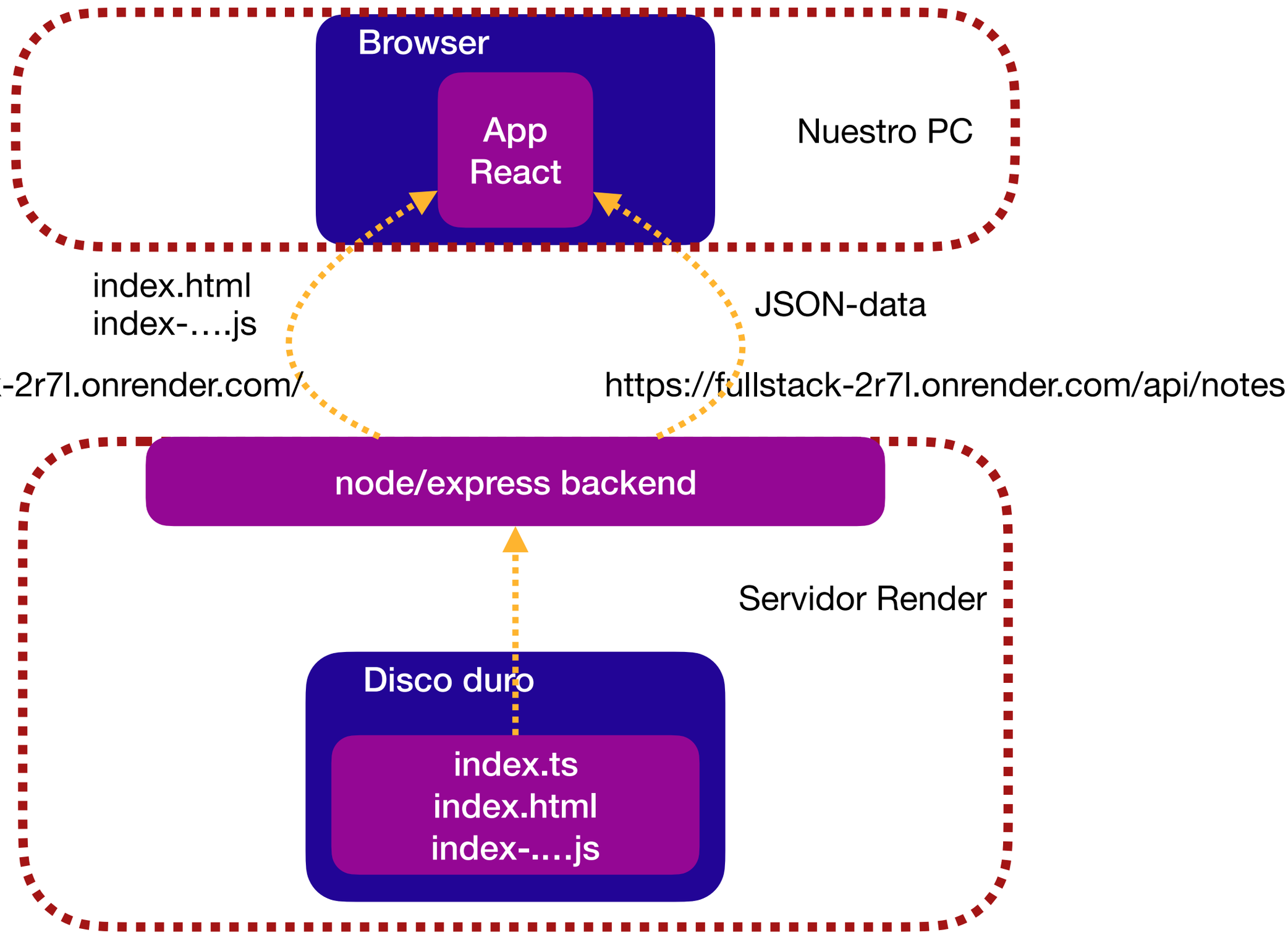
El ambiente de desarrollo



Sirviendo archivos estáticos desde el backend



El ambiente de producción



Simplificando el despliegue del frontend

package.json

```
"scripts": {
  …
  "build:ui": "rm -rf dist && cd ../frontend && npm run build && cp -r dist ../backend",
  "deploy:full": "npm run build:ui && git add . && git commit -m uibuild && git push"
 },
```

- **`npm run build:ui`:**
  - Borra el contenido de `dist`
  - Anda a la carpeta `frontend`
  - Compila los archivos
  - Copia los archivos de producción al directorio `backend`

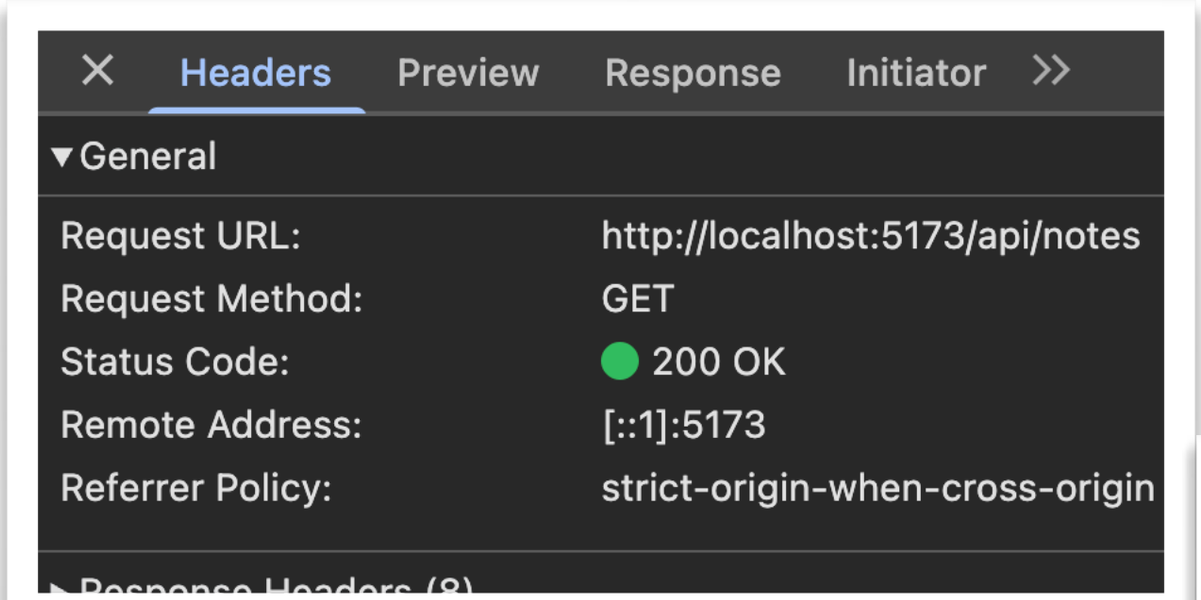
Simplificando el despliegue del frontend

package.json

```
"scripts": {
  …
  "build:ui": "rm -rf dist && cd ../frontend && npm run build && cp -r dist ../backend",
  "deploy:full": "npm run build:ui && git add . && git commit -m uibuild && git push"
 },
```

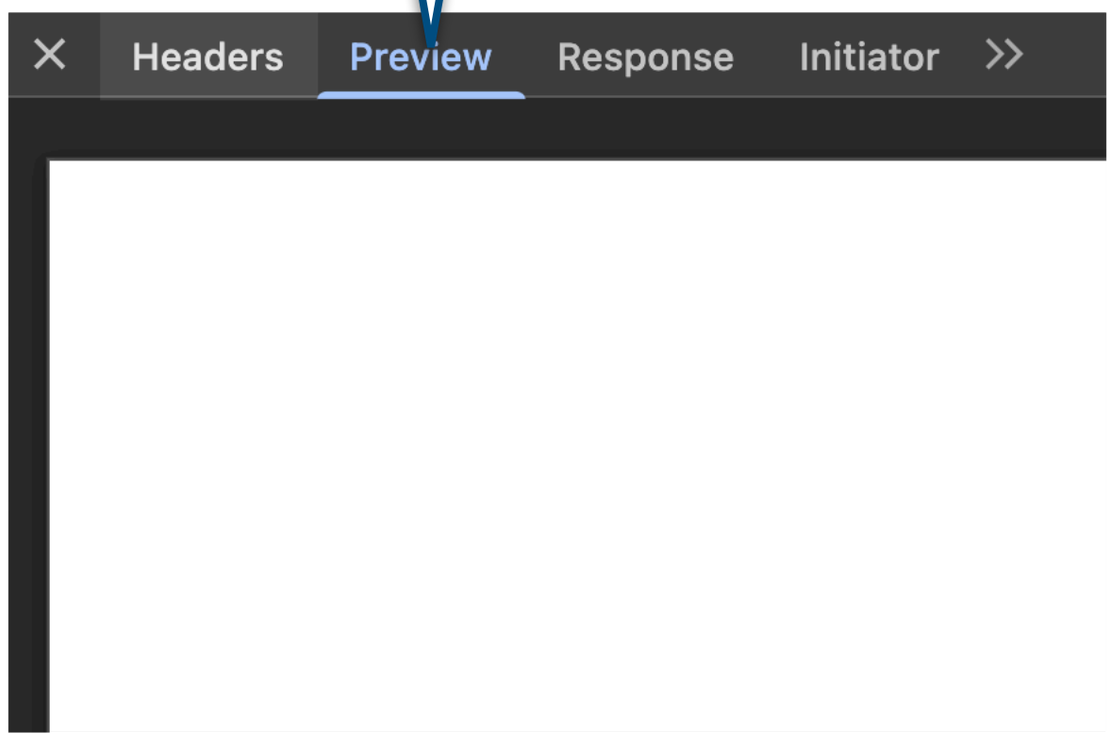
- **`npm run deploy:full`:**
  - Compila y copia el frontend al backend
  - Agrega todos los archivos nuevos/modificados a git
  - Haz commit de los cambios actuales
  - Haz push (esto lanzará un re-deploy automáticamente en Render

Proxy



El frontend **en desarrollo** no funciona porque está tratando de conectarse al backend usando una dirección relativa a su misma hostname

`http://localhost:5173/api/notes`



Pero el backend en desarrollo corre en el puerto 3001!

Proxy

vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:3001',
        changeOrigin: true,
      },
    }
  },
})
```

Esto se arregla fácilmente desde el archivo `vite.config.ts`. Le decimos que todas solicitudes que partan con `/api` lo hagan a la url del backend de desarrollo

Eliminando CORS

```
npm remove cors
```

index.ts

```ts
const app = express();

app.use(express.json());
app.use(express.static("dist"));
```

Podémoslos eliminar CORS ahora del archivo `index.ts` y de las dependencias

Guardando datos a Mongo

Instalando Mongoose

Asumiremos que Mongo esta instalado

```
npm install mongoose
npm install @types/mongoose
```

Definiendo el Schema

Nos conectamos a un mongo local por ahora. Para especificar usuario y contraseña usamos la url "mongodb://{usuario}:{contraseña}@127.0.0.1:27017/notesdb"

```
mongoose.set("strictQuery", false);
mongoose.connect("mongodb://127.0.0.1:27017/notesdb");



const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean,
});


const Note = mongoose.model("Note", noteSchema);
```

Definimos el schema como un unico documento que tiene un contenido de tipo String, y un campo important de tipo booleano.

Creamos el modelo Note usando el Schema

Obteniendo todas las notas

En ves de devolver nuestro JSON “en duro”, buscamos las notas que estén en la base de datos.

```
app.get("/api/notes", (request, response) => {
  Note.find({}).then((notes) => {
    response.json(notes);
  });
});
```

Buscamos todo ({}) y “luego”, el resultado es devuelto como JSON. Las notas importante se obtienen pasando `{important: true}`

Noten que cada objeto JSON incluye un id único.

localhost:5173


```
[
  {
    "_id": "6807d0dc1e22d86db663c7db",
    "content": "Esta es una nota en mongodb",
    "important": false,
    "__v": 0
  }
]
```

Obteniendo todas las notas

Podemos cambiar el JSON que devolvemos por defecto de la siguiente forma

```
noteSchema.set("toJSON", {
    transform: (
        document,
        returnedObject: {
            id?: string | undefined;
            _id?: mongoose.Types.ObjectId;
            __v?: number;
        },
    ) => {
        returnedObject.id = returnedObject._id?.toString();
        delete returnedObject._id;
        delete returnedObject.__v;
    },
});
```

Creando notas

Preparando el frontend

Dado que mongodb usa un id implícito por cada documento no nos preocupamos de él al crear una nota

```
const addNote = (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    const noteObject: NoteData = {
      content: newNote,
      important: Math.random() < 0.5,
      id: notes.length + 1,
    };

    noteService.create(noteObject).then((returnedNote) => {
      setNotes(notes.concat(returnedNote));
      setNewNote("");
    });
};
```

Creando notas

Preparando el frontend

types/data.ts

```
export interface NoteData {
  id: string;
  content: string;
  important: boolean;
}
```

Dado que el id implicito de mongo es un string, cambiamos el tipo a string

Usamos `Omit` para product un nuevo tipo que no tiene `id`

services/notes.tsx

```
const create = (newObject: Omit<NoteData, "id">) => {
  const request = axios.post(baseUrl, newObject);
  return request.then((response) => response.data);
};
```

Creando notas

Preparando el backend

```
app.post("/api/notes", (request, response) => {
    const body = request.body;

    if (!body.content) {
        response.status(400).json({
            error: "content missing",
        });
    } else {
        const note = {
            content: body.content,
            important: body.important || false,
        };
        const noteToSave = new Note(note);
        noteToSave.save().then((savedNote) => {
            response.json(savedNote);
        });
    }
});
```

Creamos la `Note`, y llamamos a `save()` para guardarla en la base de datos

Moviendo la configuración de la bd a su propio módulo

models/notes.ts

```
import mongoose from "mongoose";

const url = process.env.MONGODB_URI;

mongoose.set("strictQuery", false);

if (url) {
  mongoose.connect(url).catch((error) => {
    console.log("error connecting to MongoDB:", error.message);
  });
}

const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean,
});

export const Note = mongoose.model("Note", noteSchema);

noteSchema.set("toJSON", {
  transform: (document, returnedObject) => {
    returnedObject.id = returnedObject._id.toString();
    delete returnedObject._id;
    delete returnedObject.__v;
  },
});
```

Leemos ahora la url del ambiente

Si no podemos conectarnos imprimimos un error

Moviendo la configuración de la bd a su propio módulo

```
import { Note } from "./models/notes";

const app = express();

…
```

Importamos el módulo para usar `Note`

Podémoslos definir la variable de ambiente y correr el servidor así por ahora

```
MONGODB_URI="mongodb://127.0.0.1:27017/notesdb" npm run dev
```

Usando dotenv

Podemos usar la libreria `dotenv` para manejar de mejor manera las variables de entorno

```
npm install dotenv
```

Creamos un archivo `.env` en la raíz del proyecto

```
MONGODB_URI=mongodb://127.0.0.1:27017/notesdb
PORT=3001
```

Agregamos el archivo `.env` a los archivos ignorados por git

```
...
.env
```

Usando dotenv

Debemos importar `dotenv` antes de importar `Note` para que se carguen las variables de ambiente

index.ts

```ts
import dotenv from "dotenv";
dotenv.config();
import express, { NextFunction, Request, Response }
from "express";
import { Note } from "./models/notes";

const app = express();
…
const PORT = process.env.PORT;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Obteniendo una nota individual

Preparando el backend

```
app.get('/api/notes/:id', (request, response) => {
  Note.findById(request.params.id)
    .then(note => {
      if (note) {
        response.json(note)
      } else {
        response.status(404).end()
      }
    })
    .catch(error => {
      console.log(error)
      response.status(500).end()
    })
})
```

Si la nota es null devolvemos error 404

Si hay cualquier otro problema, devolvemos error 500

Moviendo *handlers* errores al *middleware*

Recibimos un nuevo parámetro `next` que representa el próximo middleware a ejecutar

```
app.get("/api/notes/:id", (request, response, next) =>
{
  Note.findById(request.params.id)
    .then((note) => {
      if (note) {
        response.json(note);
      } else {
        response.status(404).end();
      }
    })
    .catch((error) => next(error));
});
```

Llamamos el siguiente middleware pasando el `error` como parámetro

Moviendo *handlers* errores al *middleware*

Este es el último middleware que se tiene que agregar. Tiene que ir después de cada ruta

```
const errorHandler = (
  error: { name: string; message: string },
  request: Request,
  response: Response,
  next: NextFunction
) => {
  console.error(error.message);

  if (error.name === "CastError") {
    response.status(400).send({ error: "malformatted id" });
  }
  next(error);
};

app.use(errorHandler);
```

Eliminando una nota

Preparando el backend

```
app.delete("/api/notes/:id", (request, response, next) => {
  Note.findByIdAndDelete(request.params.id)
    .then(() => {
      response.status(204).end();
    })
    .catch((error) => next(error));
});
```

Actualizando una nota

Preparando el backend

```
app.put("/api/notes/:id", (request, response, next) => {
  const { content, important } = request.body;

  Note.findById(request.params.id)
    .then((note) => {
      if (note) {
        note.content = content;
        note.important = important;

        note.save().then((updatedNote) => {
          response.json(updatedNote);
        });
      } else {
        response.status(404).end();
      }
    })
    .catch((error) => next(error));
});
```

Validando datos

```
app.post("/api/notes", (request, response) => {
  const body = request.body;

  if (!body.content) {
    response.status(400).json({
      error: "content missing",
    });
  } else {
   …
});
```

Si no viene el contenido entonces devolvemos un error 400

Validando datos

```
const noteSchema = new mongoose.Schema({
  content: {
    type: String,
    minLength: 5,
    required: true,
  },
  important: Boolean,
});
```

Podemos usar mongoose para indicar validaciones especiales

Validando datos

```
app.post("/api/notes", (request, response, next) => {
  const body = request.body;

  if (!body.content) {
    response.status(400).json({
      error: "content missing",
    });
  } else {
    const note = {
      content: body.content,
      important: body.important || false,
    };

    const noteToSave = new Note(note);
    noteToSave
      .save()
      .then((savedNote) => {
        response.json(savedNote);
      })
      .catch((error) => next(error));
  }
});
```

Llamamos a `next` si es que hay un error después de guardar

Validando datos

```
const errorHandler = (
  error: { name: string; message: string },
  request: Request,
  response: Response,
  next: NextFunction
) => {
  console.error(error.message);

  if (error.name === "CastError") {
    response.status(400).send({ error: "malformatted id" });
  } else if (error.name === "ValidationError") {
    response.status(400).json({ error: error.message });
  }
  next(error);
};
```

Podemos expandir el handler de errores para incluir el caso de errores por validación

Validando datos

POST http://localhost:3001/api/notes/ Send

Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
{
    "content": "lol",
    "important": true
}
```

Body Cookies Headers (7) Test Results 400 Bad Request 37 ms 360 B

{} JSON Preview Visualize

```
{
    "error": "Note validation failed: content: Path `content` (`lol`) is shorter than the minimum
        allowed length (5)."
}
```

¿PREGUNTAS?

https://imgur.com/nother-loop-i-made-drive-0m0v4gV